

Jordi Baylina
@jbaylina



Realtime Proving & Synchronous Composability Between Rollups

DAPPCon - June 16th 2026

@ziskvm

WHAT'S ZISK?

Zisk is a 64 bit RISC-V zkVM.

Zisk lets you cryptographically prove the execution of high-level programs (Rust, Golang, C#, etc.) seamlessly.



Non-negotiable security

- 128 bits
- Provable security
- Quantum resistant
- (WIP) Auditing & Formal Verification



Fully open source

- Apache 2.0 & MIT
- Built in house



Scalable & performant

- Fast set of precompiles
- Fast proof aggregation
- Distributed proof computation.
- GPU accelerated

We are releasing
ZisK 1.0 Alpha!

ZisKVM is ready to be used and scrutinized



Code freezing

- Stable code to build and reason about.



Start auditing

- A stable code base for external scrutiny.
- Internal audit of the code.
- AI driven bug finding.
- Work towards formal verification



Developer UX in place

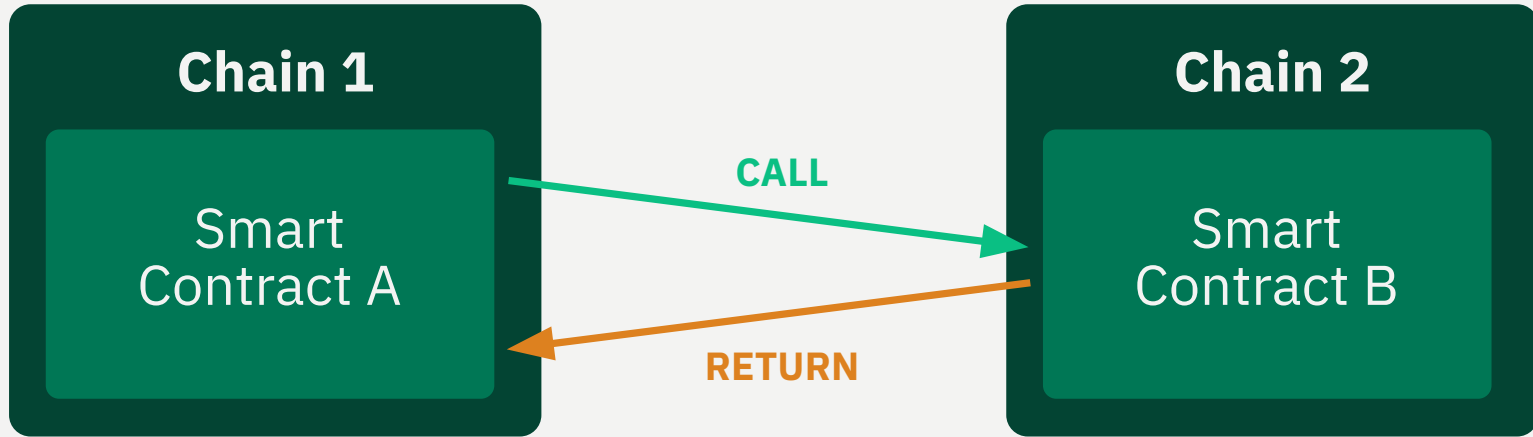
- Tooling in place: CLI, API, SDK.
- Documentation in place.
- Ensure backwards compatibility

We are also introducing
ziskethone!

The EEZ Proof

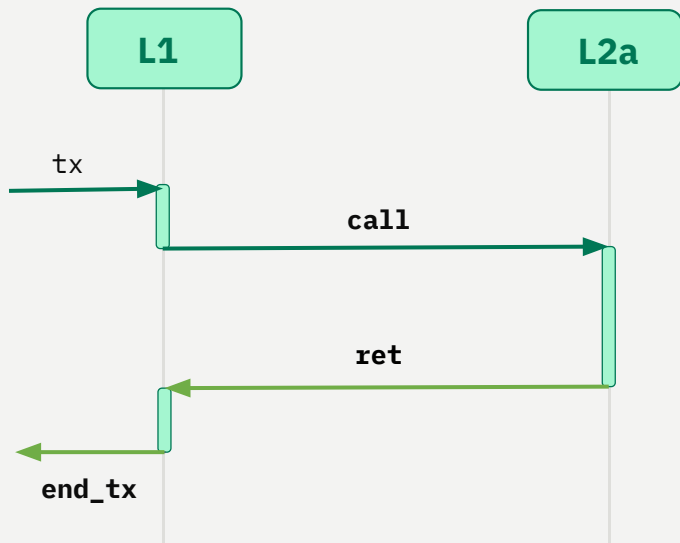
What is EEZ?

EEZ proves the combined execution of many rollups as a single, synchronous transaction.



EEZ Trace; The blob format

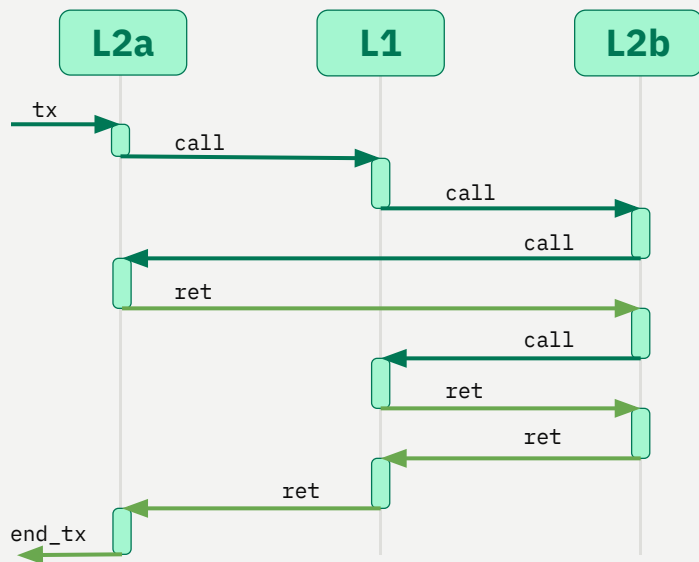
It records each context switch between chains (call and returns).



type	from	to	info
tx	L1		rawTX
call	L1	L2a	from, to, value, data
return	L2a	L1	success, retData
end_tx	L1		success

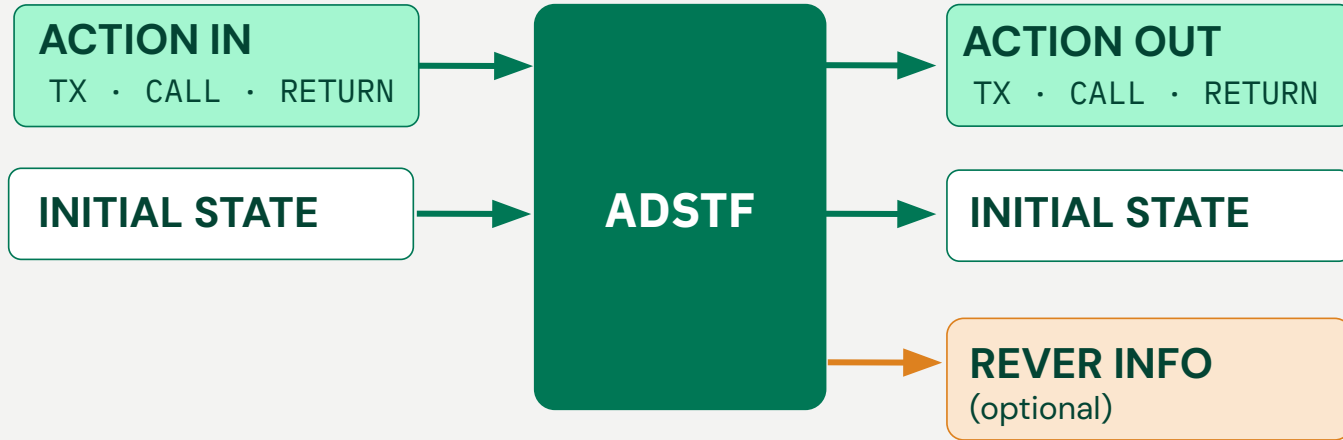
EEZ Trace: Across rollups

The trace records every nested call and return.

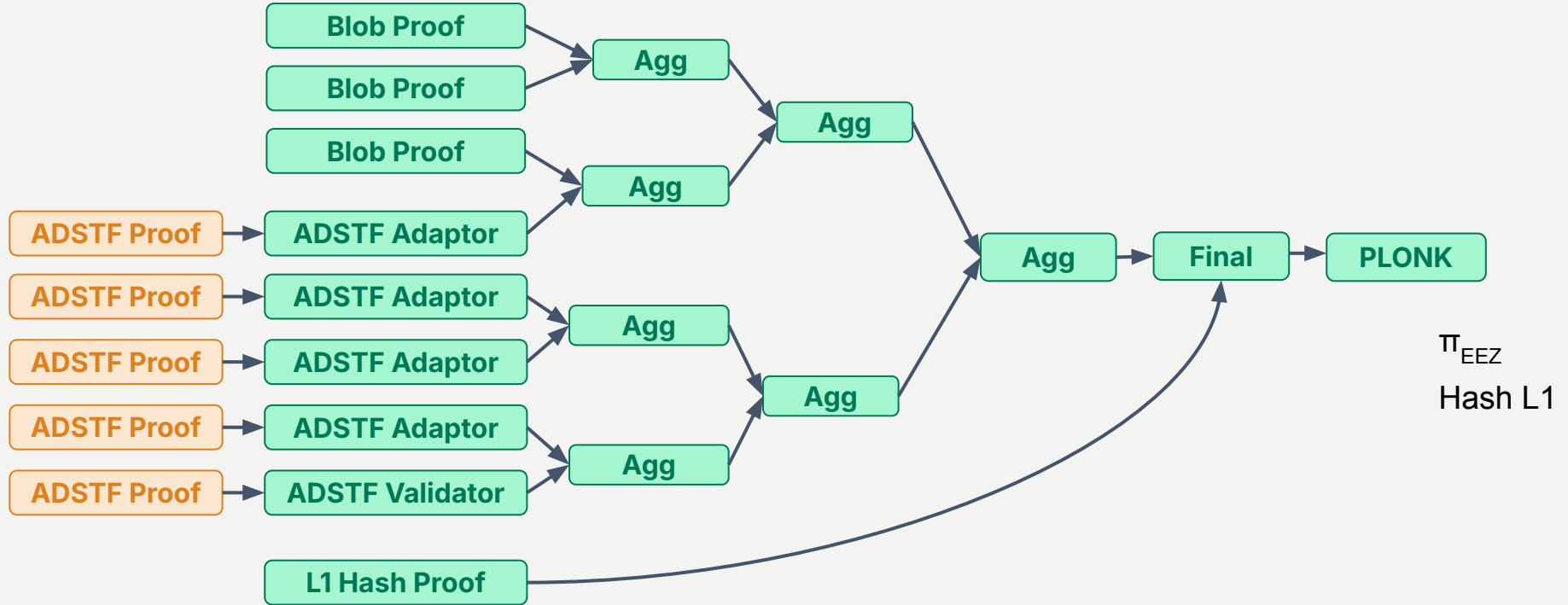


type	from	to	info
tx		L2a	rawTX
call	L2a	L1	from, to, value, data
call	L1	L2b	from, to, value, data
call	L2b	L2a	from, to, value, data
ret	L2a	L2b	success, retData
call	L2b	L1	from, to, value, data
ret	L1	L2b	success, retData
ret	L2b	L1	success, retData
ret	L1	L2a	success, retData
end_tx	L2a	-	success

Action-Driven State Transition Function



EEZ — the Zisk proof



Aggregating proofs with the cryptographic accumulator

- It can be seen as a Hash of a set.
- Sets can be added/subtracted easily.

Implementation in EEZ

$$A = \sum_{i=1}^n s_i \cdot H(e_i) \quad \text{with } s_i \in \{+1, -1\} \text{ and } n \leq N_{\max}$$

large-hash summation with a bounded element count

BLOB Processor: Parses BLOB data

PROVE	-	Blob commitment
ASUME	+	Starting blob rollup state
PROVE	-	Ending blob rollup state
ASUME	+	ADSTF
ASUME	+	RollupId → PS, VK
ASUME	+	L1 Interactions

ADSTF Adaptor: Verifies a user defined circuit VK

PROVE

-

ADSTF

L1 Hash Builder: Build the L1 Hash

ASUME	+	Blob commitment
PROVE	-	Starting blob rollup state
ASUME	+	Ending blob rollup state
PROVE	-	L1 Interactions
PROVE	-	RollupId → PS, VK

The Aggregation Circuit

- (1) Verifies 2 circuits (a basic circuit or himself)
- (2) Adds the accumulator of both circuits.

The Final Circuit

- (1) Verifies the root aggregation**
- (2) Verifies the Hash L1 bulder circuit**
- (3) Adds the accumulator of both circuits and check the sum = 0**

The PLONK Circuit

- (1) Verifies the final circuit
- (2) Generates a PLONK proof that's easily verifiable onchain

Synchronous Composability

Minimizing the proof generation latency

←..... <3s→

Bundle Building (composer)

Proof Building

WHERE WE'RE HEADED

Roadmap

- 01 **Stabilize the code**
- 02 **Audit & formal verification**
- 03 **Improve documentation & AI skills**

Future topics we are addressing

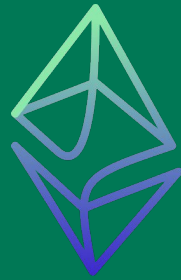
- **Move Poseidon1 to Blake**
- **ZisK BE**
- **Hints Improvement**
- **Precompiles calling overhead**
- **Wasm for Zisk**

Jordi Baylina
@jbaylina



Build without compromise

We're here to support you every step of the way.



ETHEREUM
ECONOMIC
ZONE

**Next wednesday we are
doing a deep dive
workshop into
EEZ core internals.**